

OOP

UNIT 11 – SMART POINTERS: UNIQUE_PTR, SHARED_PTR, AND MOVE SEMANTICS

Choose one owner when possible; share ownership only when the design truly requires it.

WHY THIS UNIT?

Earlier units used raw pointers to study dynamic allocation, destructors, the Rule of Three, inheritance, and run-time polymorphism. A raw pointer can locate an object, but its type does not say who must destroy that object.

This unit introduces two standard-library smart pointers. `unique_ptr` represents one owner. `shared_ptr` represents several owners that jointly keep one object alive. Move semantics explains how an exclusive owner can transfer responsibility without copying the managed object.

The scope is deliberately focused: move semantics, `unique_ptr`, `shared_ptr`, and the decision between them.

LEARNING OBJECTIVES

By the end of this unit, you should be able to:

- Explain ownership and why a raw pointer does not express deletion responsibility.
- Distinguish copying an object from moving a resource or ownership handle.
- Use `std::move` deliberately and describe the state of a moved-from smart pointer.
- Create, access, transfer, pass, and return `std::unique_ptr` objects.
- Create and copy `std::shared_ptr` objects and explain their shared lifetime.
- Choose `unique_ptr` or `shared_ptr` from the ownership requirements of a design.

OWNERSHIP AND AUTOMATIC CLEANUP

An owning pointer is responsible for the lifetime of a dynamically allocated object.

Smart pointers follow RAII: resource acquisition is initialization. The smart-pointer object acquires ownership when it is initialized and releases that ownership in its destructor.

Cleanup therefore follows ordinary C++ scope and destructor rules.

```
#include <memory>
void playPreview()
{
    auto song = std::make_unique<Song>("Imagine", 183);
    song->play();
} // song is destroyed; the Song is destroyed automatically
```

There is no explicit delete. The local `unique_ptr` is destroyed at the closing brace, and its destructor destroys the owned `Song`.

THE TWO OWNERSHIP MODELS

- `unique_ptr<T>`: exactly one smart pointer owns the object at a time.
- `shared_ptr<T>`: several smart pointers may own the same object at the same time.

Prefer `unique_ptr` : exclusive ownership is simpler and cheaper. Select `shared_ptr` only when independent parts of the program must keep the same object alive.

MOVE SEMANTICS: THE MOTIVATION

Copying creates another independent value. Moving transfers resources from one object to another. A move is especially useful when an object owns a resource that should not be duplicated, such as dynamic memory or exclusive ownership of another object.

```
std::string first = "A long title";

std::string copy = first;
                    // copy: both strings keep a value

std::string moved = std::move(first);

                    // move: moved receives the resource
```

After the copy, first is unchanged. After the move, moved contains the transferred value and first remains a valid object, but its exact value should not be assumed. It may be assigned a new value or safely destroyed.

AN INTUITIVE VIEW OF LVALUES AND RVALUES

An **lvalue** is an expression that identifies a persistent object, usually one with a name.

An **rvalue** is a temporary value or a value that may be used as the source of a move.

```
Song song("Imagine", 183); // song is an lvalue
Song another = song;      // copy from an lvalue
Song temporary("One", 217); // the constructor creates an object
Song moved = std::move(song); // allow song to be a move source
```

For this unit, the practical rule is enough: a *named* variable is treated as an *lvalue*, even when we intend to give up its resource. `std::move` marks that named variable as eligible to be moved from.

WHAT STD::MOVE ACTUALLY DOES

`std::move` does not move bytes by itself. It converts an expression into a form that allows a move constructor or move assignment operator to be selected. The selected operation performs the transfer.

```
#include <utility>

std::unique_ptr<Song> first =
    std::make_unique<Song>("Imagine", 183);

std::unique_ptr<Song> second = std::move(first);
```

The `unique_ptr` move constructor transfers its stored pointer to `second` and makes `first` empty. The `Song` is not copied and does not change its address.

COPY OPERATIONS AND MOVE OPERATIONS

A class may define four related operations in addition to its destructor:

```
Resource(const Resource& other);           // copy constructor
Resource& operator=(const Resource& other); // copy assignment

Resource(Resource&& other);                 // move constructor
Resource& operator=(Resource&& other);      // move assignment
```

The double ampersand denotes an rvalue reference. Copy operations read from their source and leave it unchanged. Move operations may transfer resources from their source.

We do not implement a resource-managing class in this unit. The important connection is that standard smart pointers already implement the correct move behavior for their ownership policy.

THE MOVED-FROM STATE

A moved-from standard-library object is valid but its value may be unspecified. It may be destroyed, assigned to, or used by operations that do not require a particular previous value.

For `unique_ptr` the result is stronger and easier to observe: after a successful move, the source `unique_ptr` is empty.

```
auto first = std::make_unique<Song>("Imagine", 183);
auto second = std::move(first);

if (!first)
    std::cout << "first is empty" << std::endl;

first = std::make_unique<Song>("One", 217); // usable again
```


UNIQUE_PTR: ONE OWNER

`std::unique_ptr<T>` represents exclusive ownership. It is the preferred smart pointer when one object or one component is clearly responsible for the lifetime of a dynamically allocated object.

```
#include <memory>

std::unique_ptr<Song> song = std::make_unique<Song>("Imagine",
183);
```

`make_unique` constructs the `Song` and returns the `unique_ptr` that owns it. Prefer `make_unique` to writing new directly because construction and ownership are established together.

ACCESSING AND CHECKING A UNIQUE_PTR

```
auto song = std::make_unique<Song>("Imagine", 183);
song->play();
std::cout << song->getTitle() << std::endl;
if (song)
    std::cout << "A Song is owned" << std::endl;
```

The *arrow* operator accesses members of the managed object. A smart pointer can be tested as a condition: true means it currently owns an object; false means it is empty.

WHY UNIQUE_PTR CANNOT BE COPIED

```
auto first = std::make_unique<Song>("Imagine", 183);  
  
// auto second = first; // error: unique_ptr cannot be copied
```

If copying were allowed, two `unique_ptr` objects would both appear to be the only owner. Exclusive ownership would lose its meaning, and both might attempt to destroy the same `Song`. Therefore the copy constructor and copy assignment operator of `unique_ptr` are deleted.

TRANSFERRING UNIQUE OWNERSHIP

```
auto first = std::make_unique<Song>("Imagine", 183);  
auto second = std::move(first);  
  
second->play();  
// first->play(); // wrong: first is empty after the move
```

The move changes ownership, not the `Song`. `second` becomes the owner; `first` becomes empty. Use `std::move` only when the source should give up ownership.

PASSING UNIQUE_PTR TO A FUNCTION

If a function only needs to use the Song temporarily, pass the Song by reference. Ownership remains with the caller:

```
void printSong(const Song& song)
{
    std::cout << song.getTitle() << std::endl;
}

printSong(*ownedSong);
```

If a function must become the new owner, accept unique_ptr by value and move ownership at the call:

```
void setFeatured(std::unique_ptr<Song> song)
{
    featuredSong = std::move(song);
}

setFeatured(std::move(ownedSong));
```

RETURNING UNIQUE_PTR

```
std::unique_ptr<Song> createSong(const std::string& title,
                                int duration)
{
    return std::make_unique<Song>(title, duration);
}

auto song = createSong("Imagine", 183);
```

A returned `unique_ptr` transfers exclusive ownership to the caller. No explicit `std::move` is needed in this return statement. This design is common for factory functions.

UNIQUE_PTR AND POLYMORPHISM

```
std::unique_ptr<MediaItem> item =
    std::make_unique<Song>("Imagine", 183);

item->play(); // virtual dispatch calls Song::play
```

The `unique_ptr` owns through the base-class type while the object is a `Song`. `MediaItem` must have a virtual destructor so destruction through the base pointer destroys the complete derived object correctly.

SHARED_PTR: SEVERAL OWNERS

`std::shared_ptr<T>` represents shared ownership. Copying a `shared_ptr` creates another owner of the same object. It does not copy the managed object.

```
#include <memory>

auto album = std::make_shared<Album>("Classics");
std::shared_ptr<Album> playerView = album;
std::shared_ptr<Album> libraryView = album;
```

All three `shared_ptr` objects refer to the same `Album`. A modification through one pointer is visible through the others because there is only one `Album` object.

HOW SHARED LIFETIME WORKS

The shared ownership group maintains an ownership count. Each owning `shared_ptr` contributes to that count. The Album is destroyed when the last owning `shared_ptr` releases it or is destroyed.

```
auto album = std::make_shared<Album>("Classics"); // 1 owner
{
    auto another = album;                          // 2 owners
    another->addSong(Song("Imagine", 183));
} // another is destroyed; 1 owner remains

album.reset(); // no owners remain; Album is destroyed
```

The exact count is mainly an explanation tool. Program logic should normally be based on the intended ownership relationships, not on repeatedly testing `use_count()`.

COPYING AND MOVING SHARED_PTR

```
auto first = std::make_shared<Album>("Classics");

auto second = first;           // copy: another owner
auto third = std::move(second); // move: ownership handle transfers

// first and third own the Album; second is empty
```

Copying a `shared_ptr` adds an owner. Moving a `shared_ptr` transfers one ownership handle and leaves the source empty; it does not add another owner. The Album itself is neither copied nor moved.

PASSING SHARED_PTR TO A FUNCTION

Pass `shared_ptr` by value when the function must keep its own ownership share after the call:

```
void rememberAlbum(std::shared_ptr<Album> album)
{
    savedAlbum = std::move(album);
}

rememberAlbum(currentAlbum); // currentAlbum remains an owner
```

If the function only needs temporary access, pass `Album&` or `const Album&` instead. This avoids implying that the function participates in ownership.

UNIQUE_PTR VS. SHARED_PTR

- Number of owners: `unique_ptr` has one; `shared_ptr` may have several.
- Copying: `unique_ptr` cannot be copied; `shared_ptr` can be copied to add an owner.
- Moving: both pointer types can be moved; the source becomes empty.
- Destruction: `unique_ptr` destroys when its owner releases it; `shared_ptr` destroys after the last owner releases it.
- Default choice: prefer `unique_ptr`; use `shared_ptr` only for genuine shared lifetime.

DECISION PROCESS

- First ask whether dynamic allocation is actually necessary. A direct object is often simpler.
- If one component controls the lifetime, use `unique_ptr`.
- If several independent components must keep the same object alive, use `shared_ptr`.
- For temporary use without ownership, pass the managed object by reference or pointer.
- Use `std::move` only when an ownership handle should leave its source.

COMMON BEGINNER MISTAKES

- Using `shared_ptr` automatically because it is easier to copy.
- Trying to copy a `unique_ptr` instead of transferring it with `std::move`.
- Dereferencing a `unique_ptr` or `shared_ptr` after it has been moved from.
- Passing a smart pointer when a function only needs a reference to the object.
- Assuming `std::move` moves an object by itself rather than enabling a move operation.
- Assuming that copying `shared_ptr` copies the managed object.
- Forgetting a virtual destructor when a smart pointer owns a derived object through a base type.

GUIDED TRACE EXERCISE

```
auto a = std::make_unique<Song>("A", 180);  
auto b = std::move(a);  
  
auto x = std::make_shared<Album>("X");  
auto y = x;  
auto z = std::move(y);
```

After each statement, identify which pointers are empty, which pointers are owners, and how many actual Song and Album objects exist. Explain why moving b would differ from copying x.

SUMMARY

Move semantics transfers resources instead of duplicating them. `std::move` marks a named object as eligible to be moved from; the selected move operation performs the transfer. A moved-from smart pointer is empty and may be assigned a new value or destroyed.

`unique_ptr` represents one owner and cannot be copied. `shared_ptr` represents shared lifetime and can be copied to add another owner. Both can be moved. Prefer `unique_ptr` when one owner is sufficient, and use `shared_ptr` only when several independent owners are part of the design.

REVIEW QUESTIONS

1. What information about ownership is missing from an ordinary raw pointer type?
2. What is the difference between copying a resource and moving it?
3. What does `std::move` do, and what does it not do?
4. What operations are safe on a moved-from object?
5. Why is `unique_ptr` not copyable?
6. What happens to the source `unique_ptr` after a move?
7. When should a function accept `unique_ptr` by value?
8. Why is returning `unique_ptr` useful for a factory function?
9. What happens when a `shared_ptr` is copied?
10. When is an object managed by `shared_ptr` destroyed?
11. How does moving a `shared_ptr` differ from copying it?
12. Why should `shared_ptr` not be the default choice?
13. When should a function receive `const T&` instead of a smart pointer?
14. Why is a virtual destructor required for polymorphic destruction?
15. Which ownership model would you choose for one clear owner, and why?